# Reading: Normalization basics

## Objectives

- Spot repeated or multi-valued data in a table and describe the update, insert or delete anomaly it causes
- Split such a table into related tables so that each fact is stored once and linked by keys

## Teaching

### Start with a guess

The library keeps every loan in one sheet, shown on the right. Guess the cost of one small change.

### Repeated facts drift apart

Ana's email appears once per loan. It is one fact stored in two places, so every change must reach every copy, in exactly the same way.

Miss one copy and the table contradicts itself: which email is right? This is an update anomaly.

### Three anomalies and a crowded cell

- Update: one fact in many rows must change everywhere.
- Insert: you cannot store one fact without another, such as a member with no loan.
- Delete: removing one row also removes a fact you wanted to keep.
- Multi-valued cell: several values in one cell, like two titles. Keep one value per cell, and give a repeating group its own rows.

### Split so each fact lives once

Normalization moves each group of related facts into its own table, with a primary key, and links tables with foreign keys.

A fact about a member goes in `members`; a fact about a book goes in `books`; a fact about one loan stays in `loans`, which points at the member and the book.

### Apply it

A different sheet with the same problem.

## Model

The library's loan sheet. Fictional data; a changed cell has a double border.

| loan_id | member_name | member_email | book_title | due_on |
|---|---|---|---|---|
| 1 | Ana | ana@example.com | Kindred | 2026-11-01 |
| 2 | Ana | ana@example.com | Beloved | 2026-11-08 |
| 3 | Ben | ben@example.com | Kindred | 2026-11-03 |

Ana's email is stored twice, once per loan. Kindred's title is stored twice too.

## Recap

- A fact repeated in many rows causes update, insert and delete anomalies; a cell holding several values hides facts.
- Split the table so each fact is stored once, in the table it describes, with one value per cell.
- Link the new tables with primary and foreign keys, so nothing is lost by the split.

Next topic: CREATE TABLE, data types and constraints

## Cheat sheet

| | |
|---|---|
| Same value repeated per row | move it to the table it describes |
| Several values in one cell | one row per value, often in a new table |
| Columns like title1, title2 | a repeating group: give it its own table |
| Split tables | link them with a foreign key to the primary key |

The formal rules are called normal forms; most applications aim for third normal form. This page covers the core idea behind them, not each rule.

## Sources

Checked against these live pages on 2026-10-14. The names update, insert and delete anomaly are common teaching vocabulary; the Microsoft article describes the same problems without those names, so they are listed as unverified in this topic's review record.

- Microsoft Learn: Description of the database normalization basics
- PostgreSQL 18: 5.5 Constraints (primary and foreign keys)

- PostgreSQL 18 tutorial: 3.3 Foreign Keys

# Static interaction alternatives

## predict

Prediction (practice, not scored): Ana changes her email. How many cells must change? Options: one; two; three; none. Answer: two, one in each of Ana's two loan rows, because the email is repeated once per loan. One would leave the other copy wrong; three would also change Ben's row; none ignores the stored copies.

## explore

Change activity. Changing Ana's email in row 1 only leaves row 1 with ana@new.example and row 2 with ana@example.com: the sheet now holds two different emails for one member, an update anomaly. Changing it in every one of Ana's rows makes both copies match again. Resetting the sheet puts the old email back so you can try again.

## explore

Classification activity with answers. Changing Ana's email in one row but not the other: update anomaly. Recording a new member, Chen, who has not borrowed anything yet, is impossible without inventing a loan: insert anomaly. Deleting loan 3, Ben's only loan, also erases Ben's email: delete anomaly. One cell holding Kindred, Beloved: a multi-valued cell, two facts in one place.

## explore

Split activity with answers. member_name and member_email go to members, stored once per member. book_title goes to books, once per book. due_on goes to loans, because it describes one loan. Each loan row points at its member with member_id, a foreign key to members.id, and at its book with book_id, a foreign key to books.id.

## mcq

Apply question: course_title for CS101 repeats in 40 rows. Options: move course_title into a courses table keyed by course_code and keep course_code in enrollments as a foreign key (answer: the title is then stored once); keep it and be careful when editing (the anomalies remain); add a second column course_title_2 (a repeating group makes it worse); delete the duplicate rows (that deletes enrollments). Second try: after the split, the title of CS101 is stored in one place, the CS101 row of courses.